APCS 程式設計系列 · U10 · 實作中高級 · C++ / Python 對照版

枚舉與貪心 重點講義

暴力枚舉先估算 · 剪枝 · 位元運算 · 位元枚舉子集 · 活動選擇 · 找零錢反例 · 短工作先做

1. 暴力枚舉：先估算次數，再決定要不要寫

枚舉就是「把所有可能一個一個試」。寫之前先算：要試幾種？每種花幾步？相乘之後不超過 10^8（Python 約 10^7）才寫得下去。

例題（百錢買百雞）：公雞每隻 5 元、母雞每隻 3 元、小雞 3 隻 1 元，用 100 元剛好買 100 隻雞，有哪些買法？

C++ 百錢買百雞：三層變兩層

```cpp
for (int x = 0; x <= 20; x++)        // 公雞最多 20 隻
    for (int y = 0; y <= 33; y++) {  // 母雞最多 33 隻
        int z = 100 - x - y;         // 小雞用算的
        if (z % 3 == 0 && 5 * x + 3 * y + z / 3 == 100)
            cout << x << " " << y << " " << z << "\n";
    }
```

Python

```python
for x in range(21):          # 公雞最多 20 隻
    for y in range(34):      # 母雞最多 33 隻
        z = 100 - x - y      # 小雞用算的
        if z % 3 == 0 and 5 * x + 3 * y + z // 3 == 100:
            print(x, y, z)
```

輸入	輸出
（不需要輸入）	0 25 75 4 18 78 8 11 81 12 4 84

n 的範圍	可以枚舉的東西	次數（約）
$n \le 10$	所有排列 n!	10! 約 3.6×10^6
$n \le 20$	所有子集 2^n	2^{20} 約 10^6，再乘 n 約 2×10^7
$n \le 500$	三層迴圈 n^3	約 10^8（Python 會太慢）
$n \le 5000$	兩層迴圈 n^2	約 2.5×10^7
$n \le 2\times10^5$	排序 + 一層掃描	n log n 約 4×10^6

剪枝：不可能的就別試了

- **縮小範圍**：公雞最多 20 隻，迴圈就不用跑到 100。
- **少一層**：最後一個未知數能用算的，就不要再用迴圈試。
- **提早 break**：已經超過預算（而且越往後只會更大），就直接跳出這一層。

2. 位元運算：把整數看成一排 0 和 1

運算	例子（6 = 110、3 = 011）	規則
&（AND）	6 & 3 = 2（010）	兩個都是 1 才是 1
|（OR）	6 | 3 = 7（111）	有一個是 1 就是 1
^（XOR）	6 ^ 3 = 5（101）	不一樣才是 1；**不是次方！**
<<（左移）	1 << 4 = 16	左移 k 位 = 乘以 2^k
>>（右移）	100 >> 3 = 12	右移 k 位 = 除以 2^k（捨去）

C++ 位元運算一次看

```cpp
cout << (6 & 3) << " " << (6 | 3) << " " << (6 ^ 3) << " ";
cout << (1 << 4) << " " << (100 >> 3) << " ";
cout << ((13 >> 2) & 1) << "\n";
```

Python

```python
print(6 & 3, 6 | 3, 6 ^ 3, 1 << 4, 100 >> 3,
      (13 >> 2) & 1)
```

輸入	輸出
（不需要輸入）	`2 7 5 16 12 1`

- **(x >> i) & 1**：取出 x 的第 i 位（從右邊、第 0 位開始數）。
- **x & 1**：等於 x % 2，判斷奇偶。
- **a ^ a = 0、a ^ 0 = a**：一串數全部 XOR，成對出現的會互相抵消。

C++ 優先順序陷阱：x & 1 == 0

```cpp
int x = 4;
cout << (x & 1 == 0) << " ";      // 0：先算 1 == 0
cout << ((x & 1) == 0) << "\n";  // 1：加括號才對
```

Python

```python
x = 4
print(x & 1 == 0, (x & 1) == 0)  # & 先算
```

輸入	輸出

（不需要輸入）	C++：
	`0 1`
	Python：
	`True True`

陷阱提醒 位元運算三大陷阱

【錯】C 的 & 比 == 晚算，x & 1 == 0 會變成 x & (1 == 0)；Python 剛好相反。一律加括號最安全。

【錯】2 ^ 3 是 XOR，等於 1，不是 8。次方要用迴圈或 pow（Python 用 2 ** 3）。

【錯】C++ 的 1 << 40 用 int 算會溢位，要寫 1LL << 40；Python 整數沒有上限。

3. 位元枚舉：用 0～2^n-1 列出所有子集

n 個東西，每個「選或不選」，共 2^n 種。把 0 到 2^n-1 的每個整數 mask 看成 n 位的二進位：**第 i 位是 1，就代表第 i 個東西有選**。

C++ 有幾個子集的和剛好等於 T

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n, t;
    cin >> n >> t;
    vector<int> a(n);
    for (int &x : a) cin >> x;
    int cnt = 0;
    for (int mask = 0; mask < (1 << n); mask++) {
        int s = 0;
        for (int i = 0; i < n; i++)
            if ((mask >> i) & 1) s += a[i];  // 有選
        if (s == t) cnt++;
    }
    cout << cnt << "\n";
}
```

Python

```python
n, t = map(int, input().split())
a = list(map(int, input().split()))
cnt = 0
for mask in range(1 << n):
    s = 0
    for i in range(n):
        if (mask >> i) & 1:      # 有選
            s += a[i]
    if s == t:
        cnt += 1
print(cnt)
```

輸入	輸出
4 10 2 3 5 7	2

- 時間是 $O(2^n \times n)$：n = 20 約 2×10^7，C++ 沒問題；Python 建議 n ≦ 16 左右。
- mask = 0 是「什麼都不選」的空集合，題目若不允許空集合要記得排除。
- n 個東西有 2^n 種選法，題目的 n 很小（≦ 20）通常就是在暗示位元枚舉。

4. 貪心：活動選擇（依結束時間排序）

貪心（greedy）就是「每一步都挑現在看起來最好的」，而且選了就不回頭。經典題：一間教室、很多活動，最多能排幾個不衝突的活動？答案是：**每次選「結束時間最早」且不衝突的活動**，留給後面的時間最多。

C++ 活動選擇：依結束時間排序

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<pair<int, int>> a(n);   // (結束, 開始)
    for (auto &p : a) cin >> p.second >> p.first;
    sort(a.begin(), a.end());      // 依結束排序
    int cnt = 0, last = -1;
    for (auto &p : a) {
        if (p.second >= last) {    // 接得上
            cnt++;
            last = p.first;
        }
    }
    cout << cnt << "\n";
}
```

Python

```python
n = int(input())
acts = []
for _ in range(n):
    s, e = map(int, input().split())
    acts.append((e, s))    # (結束, 開始)
acts.sort()                # 依結束排序
cnt, last = 0, -1
for e, s in acts:
    if s >= last:          # 接得上
        cnt += 1
        last = e
print(cnt)
```

輸入	輸出
5 1 4 3 5 0 6 5 7 8 9	3

貪心規則	反例	結果
開始時間最早	[1,10]、[2,3]、[4,5]	選 1 個，最佳是 2 個
活動長度最短	[1,5]、[4,7]、[6,10]	選 1 個，最佳是 2 個
結束時間最早	找不到反例（可以證明）	正確

5. 貪心會騙人：找零錢與反例

C++ 面額 4、3、1 湊 6 元：貪心拿到 3 枚

```cpp
int coin[3] = {4, 3, 1}, n = 6, k = 0;
for (int i = 0; i < 3; i++) {
    k += n / coin[i];      // 能拿幾枚就拿
    n %= coin[i];
}
cout << k << "\n";         // 4 + 1 + 1
```

Python

```python
coin = [4, 3, 1]
n, k = 6, 0
for c in coin:
    k += n // c          # 能拿幾枚就拿
    n %= c
print(k)                 # 4 + 1 + 1
```

輸入	輸出
（不需要輸入）	3

最好的湊法是 3 + 3，只要 2 枚。先拿最大面額的貪心，在這組面額就錯了。

貪心什麼時候能用？

- 台灣硬幣 1、5、10、50：每個面額都是下一個的倍數，先拿大的一定最少，貪心正確。
- 面額不規則（4、3、1）：貪心可能錯，要改用下一單元的動態規劃。
- **用反例檢驗**：寫貪心之前，先找 3～4 筆小資料，手算最佳解，看貪心會不會輸。
- **對拍**：小資料用暴力枚舉算正確答案，和貪心的答案比對，不一樣就代表貪心錯了。

陷阱提醒

【錯】「感覺很對」就寫貪心，沒有找反例：APCS 中高級最常見的失分方式。

【錯】貪心規則選錯排序的依據（依開始時間、依長度），範例測資會過，隱藏測資會錯。

6. 貪心 + 排序：短工作先做

一台機器、n 個工作，每個工作的「完成時間」是從 0 開始到它做完的時間。要讓所有完成時間的總和最小，就**由短到長排序後依序做**。

C++ 短工作先做：完成時間總和最小

```cpp
#include <bits/stdc++.h>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<long long> t(n);
    for (auto &x : t) cin >> x;
    sort(t.begin(), t.end());      // 短的先做
    long long now = 0, total = 0;
    for (long long x : t) {
        now += x;                  // 完成時間
        total += now;
    }
    cout << total << "\n";
}
```

Python

```python
n = int(input())
t = sorted(map(int, input().split()))   # 短的先做
now = total = 0
for x in t:
    now += x                # 完成時間
    total += now
print(total)
```

輸入	輸出
4 5 2 8 1	28

為什麼是對的？交換論證

- 第 1 個工作的時間會被算 n 次（後面每個人都要等它），最後一個只算 1 次。
- 如果長的 A 排在短的 B 前面，把它們交換：B 的完成時間變早很多，A 只變晚一點，總和一定變小。
- 一直交換到「由短到長」為止，就是最佳解。證明貪心，常用的就是這種交換論證。

讀完這份，你應該要會

- □ 寫枚舉前會先估算次數，知道 $n \leq 20$ 可以枚舉子集、$n \leq 10$ 可以枚舉排列
- □ 會用縮小範圍、少一層迴圈、提早 break 來剪枝
- □ & | ^ << >> 的結果算得出來，知道 C 的 & 比 == 晚算、Python 相反
- □ 會用 mask 從 0 跑到 2^n-1 列出所有子集，用 (mask >> i) & 1 判斷有沒有選
- □ 活動選擇會依結束時間排序，也舉得出依開始時間、依長度的反例
- □ 知道面額 4、3、1 湊 6 元時貪心會錯；短工作先做能讓完成時間總和最小

有一項沒把握，就回去重看那一段，再做練習題。

看影片講解、問問題

YouTube 莊老數學教室

IG @jf_chuang

LINE 諮詢 jenfoo2021